

GRAMA-YATRI

Mobility for Rural India

Community-Powered Bus Tracker – Live ETA for Every Village

Project No.	04
Domain	Android App Development using GenAI (Kotlin)
Category	Rural Mobility / Civic Tech
Technology	Kotlin · Firebase Realtime DB · GenAI
Document Type	Full Project Specification Report

This document provides the complete product specification, requirements analysis, feature breakdown, and technical blueprint for the Grama-Yatri Android application.

1 EXECUTIVE SUMMARY

Grama-Yatri (meaning 'Village Traveller' in Kannada/Telugu) is a **crowdsourced, community-driven rural bus tracking application** built in Kotlin for Android. It solves one of the most persistent pain points for millions of rural commuters across India — the complete unpredictability of village bus schedules.

By enabling passengers to **'Ping'** the app when they board or spot a bus, the system computes live ETAs for every subsequent village on the route, creating a **community-powered live timetable**. The application leverages Firebase Realtime Database for instant updates and is designed to function on low-end Android devices with minimal data usage.

2 PROBLEM STATEMENT

THE CORE PROBLEM

Village bus schedules are entirely unpredictable. A passenger who misses the bus by as little as **2 minutes** is forced to wait **3 hours** for the next service. There is no real-time information system, no official app, and no community channel to share bus location data. Passengers physically wait at bus stops with zero knowledge of when (or whether) the bus will arrive.

AFFECTED POPULATION

- **Daily wage labourers** travelling to city factories and construction sites.
- **School and college students** dependent on a single morning bus.
- **Rural women** visiting weekly markets or healthcare centres.
- **Elderly passengers** with no alternative transport options.

These groups collectively represent hundreds of millions of Indians living beyond the reach of urban transit infrastructure.

3 THE VISION – WHAT GRAMA-YATRI DOES

Grama-Yatri is a **Shared Transit Log** — a living, breathing timetable created *by the passengers, for the passengers*. When a commuter boards a bus at the first stop, they tap **"I am on the bus"** in the app. This single action triggers a cascade of intelligence:

- The app records the **ping timestamp** and the **GPS-tagged stop**.
- Using pre-configured **average travel times** between all stops on the route, the system calculates an **Expected Arrival Time (ETA)** for every downstream village.
- All users on that route receive a **live push notification** with the updated ETA.
- Subsequent pings from other passengers on the route **refine and correct** the ETA in real-time.
- Anomalies such as **cancellations or major delays** can be reported by any user and broadcast instantly.

The result is a **Live Timetable** that improves with every bus journey and every commuter who participates — a true example of civic technology built from the ground up.

4 SCOPE OF THE APPLICATION

4.1 In-Scope

Real-time Bus Pinging	Users can report a bus arrival, departure, or passing at any stop on a configured route.
Live ETA Calculation	Algorithmic computation of expected arrival times for all stops downstream from the reported stop.
Route Visualisation	A vertical timeline/stepper view showing all villages on a route with their current status.
Cancellation & Delay Alerts	User-reported broadcast notifications for cancelled or significantly delayed services.
Source Attribution	Every report is tagged with the reporter's name (e.g., 'Reported by Ravi at 7:14 AM').
Offline Graceful Degradation	App displays last known data with a timestamp when connectivity is unavailable.
Low-Data Mode	Minimal payload design; entire route update < 2 KB per sync.
Multi-Route Support	Support for multiple bus routes within a district or block.

4.2 Out-of-Scope (Version 1.0)

- GPS tracking of the bus vehicle itself (no hardware integration required).
- Ticket booking or payment gateway integration.
- Government KSRTC / SRTC API integration (future phase).
- iOS or web application versions.
- Multilingual support beyond Kannada and English (Phase 2).

5 APP USAGE & USER FLOW

ST EP	SCREEN / ACTION	DESCRIPTION
1	Splash & Onboarding	User selects their home village and preferred route on first launch.
2	Route View (Home)	Vertical stepper showing all villages on the route. Live ETAs shown for each stop.
3	Bus Ping	User taps 'I am on the bus' or 'Bus just passed me'. Optionally adds a note.
4	ETA Broadcast	Firebase triggers push to all subscribers. Route View refreshes with updated ETAs.
5	Alert / Report	User reports cancellation or unusual delay. Shown as a banner on all users' Route View.
6	History Log	Past pings for the day are viewable — shows who reported what and when.
7	Settings	Manage home stop, notification preferences, display name for attribution.

6 FUNCTIONAL REQUIREMENTS

FR-ID	REQUIREMENT	PRIORITY
-------	-------------	----------

FR-01	The system shall allow any registered user to submit a bus ping for any stop on a subscribed route.	MUST
FR-02	Upon a ping submission, the system shall calculate the ETA for all downstream stops using average inter-stop travel times.	MUST
FR-03	The ETA update shall propagate to all active users on that route within 3 seconds.	MUST
FR-04	Each ping shall display the reporter's name and timestamp (e.g., 'Ravi, 7:14 AM').	MUST
FR-05	Users shall be able to report a 'Bus Cancelled Today' alert for their route.	MUST
FR-06	The Route View shall render a vertical timeline of all stops with colour-coded status (upcoming, near, passed).	MUST
FR-07	The app shall send a push notification when the bus is estimated ≤ 15 minutes away from the user's home stop.	MUST
FR-08	Users shall be able to view the ping history log for the current day.	SHOULD
FR-09	The system shall support multiple bus routes within a region.	SHOULD
FR-10	Users shall be able to 'upvote' a ping to confirm its accuracy.	COULD
FR-11	The app shall display a reliability score for each route based on ping frequency.	COULD
FR-12	Admin panel for route configuration (adding stops, setting average travel times).	MUST

Priority Legend: **MUST HAVE** – Core MVP requirement **SHOULD HAVE** – Important enhancement **COULD HAVE** – Desirable if time permits

7 NON-FUNCTIONAL REQUIREMENTS

Performance

- ETA updates must reach all clients within **3 seconds** of a ping submission.
- App cold-start time must not exceed **2 seconds** on a mid-range Android device.
- Route View must render with full data within **1 second** on a 2G connection.

Reliability

- System uptime target: **99.5%** (Firebase SLA-backed).
- App must display last-known data gracefully when offline, clearly marked with a timestamp.
- Data loss on ping submission must be **zero** — use Firebase offline persistence.

Scalability

- Architecture must support up to **500 concurrent users** per route without degradation.
- Firebase Realtime Database sharding strategy must be documented for future growth.

Usability

- UI must be operable by a first-time smartphone user with **no prior training**.
- All critical actions (ping, view ETA) must be reachable within **2 taps**.
- Font size minimum: **14sp** for all primary content (accessibility).

Data Efficiency

- A full route state sync must consume **less than 2 KB** of mobile data.
- Images and heavy assets are **prohibited** in the core application flow.
- Incremental updates only — do not re-fetch entire route on each ping.

Security & Privacy

- User display names are **pseudonymous** by default — no phone number visible to other users.
- Firebase Security Rules must prevent unauthorised writes to other users' ping data.
- No personally identifiable information (PII) stored beyond display name and route preference.

8 FEATURE MATRIX – MUST-HAVE VS GOOD-TO-HAVE

FEATURE	CATEGORY	CLASSIFICATION	RATIONALE
Live Bus Ping (On-board & Passing)	Core Tracking	MUST HAVE	Primary mechanic of the entire app. Without this, there is no ETA.
Computed ETA for All Downstream Stops	Core Tracking	MUST HAVE	Core value proposition. Passengers need to know when to leave home.
Vertical Route Timeline (Stepper UI)	UI/UX	MUST HAVE	Visually communicates bus position on route intuitively.
Real-time Push Notifications (15-min alert)	Notifications	MUST HAVE	Allows users to not stare at the app; notify them when action needed.
Bus Cancellation / Delay Broadcast	Alerts	MUST HAVE	Saves wasted journeys to the bus stop for cancelled services.
Reporter Attribution (Reported by Name)	Trust & Transparency	MUST HAVE	Builds community trust and accountability in crowdsourced data.
Offline Last-Known State Display	Resilience	MUST HAVE	Rural areas have spotty connectivity; app must not show blank screen.
Multi-Route Support	Scalability	MUST HAVE	A village is typically served by 2-3 routes; all must be viewable.
Admin Route Configuration Panel	Admin	MUST HAVE	Routes and average travel times must be manageable without code changes.
Ping History Log (Today's Pings)	Transparency	GOOD TO HAVE	Helps users verify past reports and assess route reliability over the day.
Ping Upvote / Confirm Feature	Trust	GOOD TO HAVE	Multiple confirmations increase ETA confidence; reduces false reports.
Route Reliability Score	Analytics	GOOD TO HAVE	Gamifies participation; shows users which routes have active communities.
Kannada / Regional Language UI	Localisation	GOOD TO HAVE	Increases accessibility for non-English speaking rural users.
Dark Mode Support	UI/UX	GOOD TO HAVE	Reduces battery drain on AMOLED screens common in budget phones.
Shareable ETA Card (WhatsApp)	Virality	GOOD TO HAVE	A shareable image card with ETA helps non-app users and drives adoption.
Weekly Timetable Heatmap	Analytics	GOOD TO HAVE	Shows historical average departure times to help passengers plan their week.
Driver-Mode Ping (Auto Broadcast)	Future	GOOD TO HAVE	If a bus driver uses the app, pings are auto-sent at each stop via geofence.
Gamification Badges (Top Reporter)	Engagement	GOOD TO HAVE	Rewards frequent contributors; sustains community participation over time.

9 TECHNICAL IMPLEMENTATION

9.1 Technology Stack

Language	Kotlin (Android SDK 26+, targeting SDK 34)
UI Framework	Jetpack Compose with Material 3 Design System
Real-time Database	Firebase Realtime Database – for instant ping propagation and ETA sync
Authentication	Firebase Anonymous Auth (no sign-up friction for rural users)
Push Notifications	Firebase Cloud Messaging (FCM) with topic-based subscriptions per route
Local Persistence	Room Database – caches last known route state for offline display
Architecture	MVVM with Clean Architecture; Kotlin Coroutines + StateFlow
Dependency Injection	Hilt (Dagger) for testability and modular structure
GenAI Integration	Gemini API (Google AI Studio) – for smart ETA confidence scoring and anomaly detection

9.2 ETA Calculation Logic

The ETA engine is a simple but effective algorithm based on a **List of Stops** with pre-configured **Average Travel Time (ATT)** between consecutive stops.

Algorithm: Given a ping at Stop *i* at time *T*, for any downstream Stop *j* where *j* > *i*:

ETA(j) = T + Σ ATT(k → k+1) for k from i to j-1

- ATT values are stored in Firebase and editable by an admin.
- When multiple pings exist for the same bus journey, the **most recent ping** takes precedence.
- GenAI (Gemini) analyses ping patterns to flag **anomalous journeys** (e.g., bus is 30 mins late vs average) and adjusts confidence bands.

9.3 Firebase Data Structure

The Firebase Realtime Database is structured for minimal read/write overhead:

```
routes/  
  {routeId}/  
    name: 'Hosapete - Bellary'  
    stops: [ {id, name, avgTimeToNext} ]  
    activePing/  
      stopIndex: 3  
      timestamp: 1719900000  
      reportedBy: 'Ravi'  
      journeyId: 'J_20240702_07'  
      alerts/  
        {alertId}: {type, message, reportedBy, timestamp}
```

10 IMPACT GOALS

Efficient Mobility	Eliminating wasted man-hours at bus stops. Even a 30-minute saving per commuter per day translates to 180+ hours reclaimed annually per person.
Economic Connectivity	Helping rural workers arrive at city jobs reliably and on time, reducing late-arrival penalties and enabling career progression.
Social Equity	Making rural transport as predictable and trustworthy as urban metro systems — closing the information gap between cities and villages.
Community Ownership	A system maintained by the community itself; no dependency on government IT infrastructure or budget allocations.

11 SUCCESS CRITERIA FOR STUDENTS

ID	TYPE	CRITERION	IMPORTANCE
SC-01	MANDATORY	The 'Live ETA' must update for all subscribed users on the route within 3 seconds of a ping being submitted.	CRITICAL
SC-02	MANDATORY	Every ping displayed in the Route View must show the source reporter's name and time (e.g., 'Reported by Ravi at 7:14 AM').	CRITICAL
SC-03	MANDATORY	The UI must be operable on a 2G connection. Total data for a full route refresh must not exceed 2 KB.	CRITICAL
SC-04	MANDATORY	A bus cancellation alert must appear on all active users' screens within 5 seconds of submission.	HIGH
SC-05	RECOMMENDED	The app must function in a degraded offline state, showing last-known data with a clear 'Last updated: X mins ago' indicator.	HIGH
SC-06	RECOMMENDED	A new user must be able to complete their first ping within 60 seconds of app installation.	MEDIUM
SC-07	STRETCH	GenAI integration (Gemini) correctly flags a ping that is statistically anomalous (e.g., bus 45 mins early) with a confidence warning.	MEDIUM

12 GENAI INTEGRATION – KOTLIN & GEMINI API

Grama-Yatri uses the **Google Gemini API** (via the Android Gemini SDK) to add an intelligence layer on top of raw crowdsourced data:

- **ETA Confidence Scoring:** Gemini analyses the historical ping pattern for a route and assigns a confidence percentage to the computed ETA. E.g., 'ETA 7:42 AM (87% confident)'.
- **Anomaly Detection:** If a ping timestamp deviates significantly from historical norms (e.g., bus is 45 minutes late), Gemini flags it and warns users: 'This report seems unusual. Treat ETA with caution.'
- **Natural Language Alerts:** Instead of raw data, Gemini converts delay/cancellation reports into human-readable notifications: 'The 7:15 AM bus from Hosapete appears to be cancelled today.'
- **Route Health Summary:** A weekly Gemini-generated summary tells users: 'This route has been 89% on-time this week. Best day: Tuesday.'

13 GLOSSARY & ABBREVIATIONS

Ping	A user-submitted report indicating the bus has been seen at or has passed a specific stop.
ETA	Estimated Time of Arrival — the computed expected arrival time of the bus at a given stop.
ATT	Average Travel Time — the historically observed average time for a bus to travel between two adjacent stops.
Route	A named sequence of village stops served by a single bus service (e.g., 'Hosapete – Bellary via Kudligi').
FCM	Firestore Cloud Messaging — Google's push notification service used for real-time alerts.
Grama-Yatri	Kannada/Telugu: 'Village Traveller'. The name of this application.
MVVM	Model-View-ViewModel — the recommended Android architecture pattern used in this project.
GenAI	Generative Artificial Intelligence — referring to the Gemini model used for intelligent ETA analysis.

This document was prepared as part of the GenAI Android Development (Kotlin) academic project series. All feature requirements are subject to revision based on mentor feedback and sprint retrospectives.